forensics. 10. Image forensics, QR, fingerprints, compare. 11. Generator and scorer (start the generator early, from day 1, because every other part depends on its documents). 12. Tests, tuning on seed 1, report on seed 42.

3. Frontend work order (React, Vite, TypeScript)

3.1 Responsibilities

Render the API and collect human input. Do no business logic: no date math, no rule evaluation, no scoring, no status derivation. Every number, status, band and date comes from an API response. Build mock-first with MSW so this work does not wait for the backend.

Dependencies: react, react-dom, react-router-dom, @tanstack/react-query, axios, zod, react-hook-form, react-pdf, recharts, lucide-react, tailwindcss, clsx, date-fns, @microsoft/fetch-event-source (SSE with an auth header), msw, vitest, @testing-library/react, @playwright/test. Optional: react-i18next, jsdiff.

3.2 Structure

```
apps/web/src/
  main.tsx  App.tsx  routes.tsx
  api/        client.ts queries.ts sse.ts   (types imported from @contracts)
  mocks/      browser.ts handlers.ts fixtures/ (faultedApplication.ts
cleanApplication.ts ...)  public/mock-docs/*.pdf
  auth/       AuthProvider.tsx RequireRole.tsx
  components/ ui/ (Button Chip Table Tabs Drawer Toast Skeleton EmptyState
ErrorState Banner)
              status/ (StatusBadge SeverityChip BandChip StateChip
PlaceholderTag DemoBanner)
              evidence/ (EvidenceViewer BBoxOverlay EvidenceCompare
RecordEvidenceCard)
              case/ (FindingList FindingDetail DispositionForm
ProofComputation PassedChecks SignalList)
              data/ (ClaimTable ClaimEditorDrawer LedgerPanel
ValidityTimeline ChecklistPanel UploadZone)
  pages/      login/ institution/ officer/ admin/ audit/ benchmark/
  hooks/      useApplication useFindings useClaims useSSE useDebounced
  styles/     tokens.css  i18n/ en.json
  tests/      unit/ e2e/
```

Environment: `VITE_API_BASE=/api`, `VITE_USE_MOCKS=true or false`. In Vite config proxy `/api` to `http://localhost:4000`. Configure the pdf.js worker with `pdfjs.GlobalWorkerOptions.workerSrc = new URL('pdfjs-dist/build/pdf.worker.min.mjs', import.meta.url).toString()`.

3.3 Design rules

- Government-service look: calm neutral surface, one accent colour, high contrast, 16 px base text, no decorative animation. Layout is responsive to 1280 px and above first; tablets must work; phones are best effort.
- **Never rely on colour alone.** Every status has an icon and a text label.
- **Language rules.** The UI never says fraud, forged or fake. Use 'possible anomaly', 'needs verification', 'mismatch detected'. Every case view carries the footer: 'Automated findings are advisory and are for review by authorized officials.'
- A `DemoBanner` is shown whenever any data has `kind: 'SYNTHETIC_DEMO'`: 'Synthetic data and simulated sources. Not an official verification.'
- `PlaceholderTag` appears next to any finding or checklist item with `placeholder: true`: 'PLACEHOLDER norm, not yet checked against the handbook'.

Status	Label	Icon (lucide)	Tone
VERIFIED	Verified	check-circle	green
MISMATCH_DETECTED	Mismatch detected	x-circle	red
EXPIRED	Expired	clock	red
VERIFICATION_PENDING	Verification pending	hourglass	blue
SOURCE_UNAVAILABLE	Source unavailable	cloud-off	grey
MANUAL_REVIEW_REQUIRED	Manual review required	eye	amber
NOT_CHECKED	Not checked	minus-circle	grey
BLOCKED_LOW_CONFIDENCE	Needs confirmation	help-circle	amber

Bands: RED 'High priority', YELLOW 'Needs attention', GREEN 'Routine'. Severity chips: INFO, LOW, MEDIUM, HIGH, CRITICAL with text. All of these render only through `StatusBadge`, `BandChip`, `SeverityChip`; no page re-implements them.

3.4 Routing and role guards

Route	Roles	Page
/login	public	Sign in
/portal	INSTITUTION	My applications
/portal/new	INSTITUTION	Create application
/portal/applications/:id	INSTITUTION	Application workspace (tabs below)
/review	SCRUTINY_OFFICER, REGIONAL_DIRECTOR	Queue and dashboard
/review/:id	same	Case packet
/review/:id/letters	same	Deficiency letters
/admin/rulepacks, /admin/checklists, /admin/users	ADMIN	Configuration
/audit	AUDITOR, ADMIN	Audit log and chain check
/benchmark	ADMIN, OFFICER	Benchmark runs

`RequireRole` redirects unauthenticated users to /login and shows a 403 screen for the wrong role. After login, route by role. The nav shows only what the role may open.

3.5 API client and state

- `axios` instance with `baseURL`, bearer token from `AuthProvider` (keep the token in memory and `sessionStorage`), `X-Request-Id` per call. Map `{ error: { code, message } }` to toasts; 401 logs out; 403 shows the forbidden screen; 409 opens a 'This item changed, reload?' dialog.
- TanStack Query keys: `['apps', filters]`, `['app', id]`, `['claims', id]`, `['findings', id]`, `['ledger', id]`, `['docs', id]`, `['alerts']`, `['audit', filters]`, `['bench', runId]`. Mutations invalidate the narrowest keys.

- Live updates: `useSSE` connects to `/api/events` with fetch-event-source and the auth header; on application events invalidate `['app', id]`, `['findings', id]`, `['ledger', id]`; on alert events invalidate `['alerts']` and show a toast. Fall back to 5-second polling if SSE fails.
- Original files: `react-pdf` with `file={{ url: '/api/documents/<id>/file', httpHeaders: { Authorization: 'Bearer ...' } }}`. Images render with an authenticated blob URL.

3.5.1 Evidence overlay math

Every `bbox` is `[x0, y0, x1, y1]` normalized to 0..1 on a 1-based page, origin top-left. Render each page inside a `position: relative` wrapper; place an overlay `div` with `left: x0*100%`, `top: y0*100%`, `width: (x1-x0)*100%`, `height: (y1-y0)*100%`. Percentages make the overlay independent of zoom. The overlay is a button with an accessible name (finding title) and the same list of overlays must exist as a keyboard-reachable list beside the viewer.

3.6 Pages

Login. Email and password; demo credentials hint when `DEMO` is on; error states for wrong credentials and rate limiting.

Institution: My applications. Table with application number, category, status chip, band (only after submission), missing items count, last updated; empty state with a 'Create application' button.

Institution: New application. Choose category (options from the API), show the checklist that will apply, create, go to the workspace.

Institution: Workspace tabs.

1. *Checklist and uploads.* One row per checklist item: label, clause reference (with PlaceholderTag when needed), required badge, state (MISSING, UPLOADED, EXTRACTING, DONE), a drop zone (`UploadZone` sends `docTypeHint`), replace and delete-version actions, per-file processing progress from SSE, and the detected document type with a 'Wrong type? Change' control. Validation before upload: mime and size match the API limits; show the server error text otherwise.
2. *Extracted data.* `ClaimTable` with predicate label, value, unit, confidence bar, status chip and a source link that opens the evidence viewer. Low-confidence rows are highlighted with a 'Please confirm' action; Confirm and Edit (value plus mandatory

reason) call the API; after a correction show a toast built from the response: 'Re-checked X of Y rules; Z resolved, W new.'

3. *Self-check.* A 'Run self-check' button; results list each finding in plain language (title, what is wrong, where, what to provide) with the clause reference. Never show officer-only notes. Submit is disabled with a tooltip listing missing required items until they exist.
4. *Messages.* Deficiency letters sent by the officer, each linked to the findings it concerns, with 'Upload or correct' actions that return to tabs 1 and 2.
5. *History.* Timeline of status changes and uploads.

Officer: Queue. Summary cards from `/dashboard/officer`: count per band, pending source verifications, oldest unresolved, average time per stage. Table with application number, institution, category, `BandChip`, open findings per severity (mini counts), pending sources, claims needing confirmation, submitted age. Filters: band, status, search. Default order is the API `sortKey`. Rows update live from SSE. Row click opens the case.

Officer: Case packet (the main screen). Three panes plus a bottom drawer.

- *Header:* application number, institution, category, band, status, source label, and buttons 'Request clarification', 'Refer for inspection', 'Record outcome' (a dialog; adverse-action recommendation requires a reason, and the button explains the server rule if blocked).
- *Left (findings):* grouped by severity; each row shows `StatusBadge`, title, rule id, state chip. Filters for status and state. A collapsible 'Checks passed (n)' lists trace entries with their compared values. A 'Document signals' section lists forensic signals per document with their limitation text.
- *Centre (evidence):* `EvidenceViewer` tabs per document, page navigation, zoom, and overlays for the selected finding's evidence. `EvidenceCompare` shows two sources side by side (for example the form and the plan); `RecordEvidenceCard` shows a record value with its source label, timestamp and the DEMO tag. Selecting an evidence line jumps to its document, page and box and pulses the overlay.
- *Right (detail):* the proof statement, `ProofComputation` (formula, inputs, result, rendered from `computation`), the evidence list, clause reference, the disposition form (Accept, Dismiss with a mandatory reason of at least 10 characters, Request clarification which queues the finding for the letter), and a per-finding history from the audit endpoint.
- *Bottom drawer tabs:* Ledger, Validity, Claims, Alerts, Audit (for this application).

Verification ledger panel. One card per document and check type: status via `StatusBadge`, source label and kind, requested and responded times, a request and response summary (expandable), and for MANUAL queue items a 'Record manual confirmation' action (outcome and note). History is collapsed under 'Earlier results'.

Validity timeline. A horizontal timeline from the API `validity[]`: each document as a marker at `validUntil`, the approval period as a shaded band, the decision date as a vertical line, states VALID, LAPSES_IN_PERIOD, EXPIRED with icon and label, days remaining as text. In DEMO mode a slider moves the decision date (debounced 400 ms) through `PUT /demo/settings`; while the API re-runs, show a spinner and then update from `validity[]` and findings. The slider is the live-demo moment, so make the transition visible: markers change state in place.

Officer: Letters. Select open findings, preview the generated letter (editable text area), save draft, send; sent letters list with timestamps. Confirm dialog before sending.

Admin: Rulepacks. Version table with active chip; read-only YAML view; 'New version' with a text area, 'Validate and save' (server errors shown as a list with rule ids), 'Activate' with a confirm dialog; optional diff versus the active version. **Checklists** and **Users** are simple CRUD tables with version history for checklists.

Audit. Filters (entity id, action, date range); virtualized table; row expand shows the payload JSON. 'Verify chain' button calls `/audit/verify` and shows a green 'Chain intact (N events checked)' or a red 'Chain broken at event #S'.

Benchmark. Form (n, seed, fault rate, scan ratio) and Run; progress by polling; results as headline cards (extraction accuracy, validation accuracy, clean-app false-positive rate, average verification time per application), a per-fault table (precision, recall, F1 with TP, FP, FN), a Recharts bar chart of recall per fault, scan versus text-layer extraction accuracy, alert timeliness, seed and manifest hash, a SYNTHETIC badge, and 'Download report (JSON)'.

Demo controls drawer (visible only when `/demo/settings` is enabled): decision-date slider, force-source-unavailable toggle, link to Benchmark, and a dev 'Tamper audit event' button that calls the dev endpoint and then triggers verify.

3.7 Mocks and fixtures

MSW handlers cover every endpoint in 1.6 with realistic data. The `faultedApplication` fixture contains the five planted findings (area mismatch, intake against record, TER mismatch, NOC lapsing within the period, edited PDF signal) with real bboxes over the

PDFs in `public/mock-docs/`. Fixtures must exercise: a BLOCKED_LOW_CONFIDENCE finding, a SOURCE_UNAVAILABLE ledger entry, a VERIFICATION_PENDING entry, an application with no findings, an empty list, and a 500 error. Example finding fixture:

```
export const areaFinding: Finding = {
  findingId: 'F-017', applicationId: 'A-100', ruleId: 'AREA-CONSIST-001',
ruleVersion: '1',
  rulepackVersion: '2026.1-demo', clauseRef: 'HANDBOOK-REF-TBD', placeholder:
true,
  status: 'MISMATCH_DETECTED', severity: 'HIGH', state: 'OPEN',
  title: 'Laboratory area differs between application form and building
plan',
  message: 'Laboratory area is 640 sqm in the form but 410 sqm in the plan
(difference 56%).',
  claimIds: ['C101', 'C238'],
  proof: {
    statement: 'lab_area_sqm_total differs between documents beyond the 2%
tolerance',
    evidence: [
      { label: 'Self-disclosure form, p.4', value: 640, unit: 'sqm',
confidence: 0.97,
        source: { docId: 'D1', page: 4, bbox: [0.18, 0.42, 0.52, 0.46],
snippet: 'Total laboratory area: 640 sq m', docType: 'SELF_DISCLOSURE_FORM' }
},
      { label: 'Building plan, p.2', value: 410, unit: 'sqm', confidence:
0.91,
        source: { docId: 'D2', page: 2, bbox: [0.55, 0.81, 0.92, 0.85],
snippet: 'Laboratories (total): 410 sqm', docType: 'BUILDING_PLAN' } },
    ],
    computation: { formula: '|form - plan| / plan * 100', inputs: { form:
640, plan: 410 }, result: 56.1 },
  },
  createdAt: '2026-10-24T09:30:00Z',
};
```

3.8 States and errors

Every data view has a skeleton, an empty state with a next action, and an error state with retry. Processing shows per-document progress rather than a spinner. Offline or SSE loss shows a slim banner and switches to polling. Forbidden and not-found screens are

distinct. Long tables are paginated or virtualized. A failed analysis shows the MANUAL_REVIEW_REQUIRED finding the backend created and a 'Retry analysis' button for officers.

3.9 Accessibility and language

Keyboard-complete case packet (tab order: findings, viewer, detail, drawer; arrow keys move through findings; Enter focuses the evidence); visible focus rings; ARIA roles and labels on overlays, tabs and dialogs; contrast at AA or better; status never colour-only; `prefers-reduced-motion` respected. Put all strings in `i18n/en.json` so a Hindi file can be added later; do not machine-translate officer-facing legal wording.

3.10 Tests and acceptance

- Vitest and React Testing Library: `StatusBadge` renders the label and icon for all 8 statuses; overlay percentage math; claim editor requires a reason; `RequireRole` blocks the wrong role; ClaimTable highlights low confidence; a banned-words check fails the build if the UI strings contain fraud, forged or fake.
- Playwright flows against mocks and, later, the real stack: (1) institution uploads, runs self-check, sees findings; (2) officer opens the faulted case, selects a finding, the viewer jumps to the cited box, dismisses with a reason; (3) moving the date slider changes a validity state; (4) auditor verifies the chain.

Done when: every route works against MSW alone; the same build works against the real API by flipping `VITE_USE_MOCKS`; every status renders through the shared components; all numbers shown come from responses; keyboard-only completion of flow (2) works; the DEMO banner and PLACEHOLDER tags appear wherever required.

3.11 Build order

1. Scaffold, tokens, layout, auth and guards with MSW. 2. Status components, API client and query hooks. 3. Institution workspace with upload and checklist. 4. `EvidenceViewer` and overlays against fixture PDFs (the most important component; build and test it early). 5. Officer queue and case packet. 6. Claim editor and the re-run toast. 7. Validity timeline, ledger panel and demo controls. 8. Letters. 9. Admin, audit and benchmark pages. 10. Accessibility pass, e2e tests, integration with the real API.